COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	7/23/2026	**DATE SUBMITTED:**	7/23/2026
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

1.

```cpp
#include <iostream>
#include "singly_ll.h"

class Node {
public:
    char data;
    Node* next;
};

int main() {
    Node* head = nullptr;
    Node* second = nullptr;
    Node* third = nullptr;
    Node* fourth = nullptr;
    Node* fifth = nullptr;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;

    head->data = 'J';
    head->next = second;

    second->data = 'I';
    second->next = third;

    third->data = 'R';
    third->next = fourth;

    fourth->data = 'Q';
    fourth->next = fifth;

    fifth->data = 'H';
    fifth->next = nullptr;

    delete head;
    delete second;
    delete third;
    delete fourth;
    delete fifth;

    return 0;
}
```

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SingleList{
    public:
    T data;
    SingleList<T> *next = nullptr;

};

#endif
```

2.

main.cpp

```cpp
#include <iostream>
#include "singly_ll.h"

int main() {
    SingleList<int> *head = new SingleList<int>;
    SingleList<int> *second = new SingleList<int>;
    SingleList<int> *third = new SingleList<int>;

    // storing the data and linking the nodes
    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = nullptr;

    std::cout << "Linked list traversal: ";
    listTraversal(head);

    delete head;
    delete second;
    delete third;

    return 0;
}
```

singly

```
#ifndef SINGLY_LL_H // header filename
#define SINGLY_LL_H
#include <iostream> // Included so std::cout and std::endl

template <typename T> // allows the class to accept any d
class SingleList {
public:
    T data; // data of the node
    SingleList<T> *next = nullptr; // stores address of t
};

template <typename T>
void listTraversal(SingleList<T> *head) {
    while (head != nullptr) {
        std::cout << head->data;

        // checks if the next node is not equal to null
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}

#endif
```

3.
main.cpp

```
#include <iostream>
#include "singly_ll.h"

int main() {
    SingleList<int> *head = new SingleList<int>;
    SingleList<int> *second = new SingleList<int>;
    SingleList<int> *third = new SingleList<int>;

    // storing the data and linking the nodes
    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = nullptr;

    std::cout << "Linked list traversal: ";
    listTraversal(head);

    std::cout << "Newly Inserted Head: ";
    sllinsertHead(0, &head);
    listTraversal(head);

    delete head;
    delete second;
    delete third;

    return 0;
}
```

```cpp
#ifndef SINGLY_LL_H // header filename
#define SINGLY_LL_H

#include <iostream> // Included so std::cout and std::endl work
template <typename T> // allows the class to accept any data type
class SingleList {
public:
    T data; // data of the node
    SingleList<T> *next = nullptr; // stores address of the next node
};
template <typename T>
void listTraversal(SingleList<T> *head) {
    while (head != nullptr) {
        std::cout << head->data;

        // checks if the next node is not equal to null
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}
template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead) {

    SingleList<T>* newNode = new SingleList <T>;

    newNode->data = newData;

    newNode -> next = *currentHead;

    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsertion(T newData, SingleList<T> *prevNode) {

    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }
    SingleList <T>* newNode = new SingleList<T>;
    newNode -> data = newData;
    newNode -> next = prevNode -> next;
    prevNode -> next = newNode;
}
#endif
```

4.

main.cpp

```cpp
#include <iostream>
#include "singly_ll.h"

int main() {
    // this line creates a empty kinked list
    SingleList<int> *head = new SingleList<int>;
    SingleList<int> *second = new SingleList<int>;
    SingleList<int> *third = new SingleList<int>;

    head -> data = 1;
    head -> next = second;

    head -> data = 2;
    head -> next = third;

    head -> data = 3;
    head -> next = nullptr;

    std::cout <<"Linked List Traversal: ";
    listTraversal(head);

    std::cout<< "Newly Inserted Head: ";
    sllInsertHead(0,&head);
    listTraversal(head);

    std::cout<< "Newly inserted after a node of linked list: \n";
    sllGeneralInsertion(3,head->next);
    listTraversal(head);

    sllGeneralInsertion(4,head->next->next);
    listTraversal(head);

    sllInsertEnd(10,&head);
    listTraversal(head);

    delete head;
    delete second;
    delete third;

    return 0;
}
```

singly

```cpp
#ifndef SINGLY_LL_H // header filename
#define SINGLY_LL_H

#include <iostream> // Included so std::cout and std::endl work

template <typename T> // allows the class to accept any data type
class SingleList {
public:
    T data; // Data of the node
    SingleList<T> *next = nullptr; // stores address of the next node
};

template <typename T>
void listTraversal(SingleList<T> *head) {
    while (head != nullptr) {
        std::cout << head->data;

        // checks if the next node is not equal to null
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}

template <typename T>
void sllInsertHead(T newData, SingleList<T>** currentHead) {
    SingleList<T>* newNode = new SingleList<T>;

    newNode ->data = newData;

    newNode -> next = *currentHead;

    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsertion(T newData, SingleList<T>* prevNode) {
    if(prevNode==nullptr){
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

    SingleList <T>* newNode = new SingleList<T>;

    newNode -> data = newData;

    newNode -> next = prevNode -> next;
```

```
50
51      prevNode -> next = newNode;
52  }
53
54  template <typename T>
55  void sllInsertEnd(T newData, SingleList<T>** head){
56
57      SingleList<T>* currentNode = *head;
58
59      //Allocate new node
60      SingleList<T>* newNode = new SingleList<T>;
61
62      //Dereference to the head node
63      SingleList<T>* currentNode = *head;
64
65      // Store data in new node
66      newNode->data = newData;
67      //Point next of new node to NULL
68      newNode->next = nullptr;
69
70      // check if the list is empty
71      if(currentNode == nullptr){
72          currentNode = newNode;
73          return;
74      }
75
76      //Traverse the list until next of the node is null
77      while(currentNode->next != nullptr){
78          //Iteration point to one node to another
79          currentNode = currentNode->next;
80      }
81
82      currentNode ->next = newNode;
83  }
84  #endif
```

5.

```cpp
#include <iostream>
#include "singly_ll.h"

int main() {
    // Create initial nodes
    singlelist<int> *head = new singlelist<int>;
    singlelist<int> *second = new singlelist<int>;
    singlelist<int> *third = new singlelist<int>;

    // Correctly link nodes together
    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = nullptr;

    std::cout << "Linked List Traversal: ";
    listTraversal(head);

    std::cout << "Newly Inserted Head: ";
    sllInsertHead(0, &head);
    listTraversal(head);

    std::cout << "Newly inserted after a node of linked list:\n";
    sllGeneralInsertion(3, head->next);
    listTraversal(head);

    sllGeneralInsertion(4, head->next->next);
    listTraversal(head);

    sllInsertEnd(10, &head);
    listTraversal(head);

    // Clean up all dynamically allocated nodes to avoid memory leaks
    singlelist<int>* temp = head;
    while (temp != nullptr) {
        singlelist<int>* nextNode = temp->next;
        delete temp;
        temp = nextNode;
    }

    return 0;
}
```

Singly

```cpp
#ifndef SINGLY_LL_H // header file name
#define SINGLY_LL_H
#include<iostream>
template <typename T> // enables the class accept any data type


class singlelist{
    public:
    T data; //data of the node
    singlelist<T> *next = nullptr; //stores address of next node

};
template <typename T>
void listTraversal(singlelist<T>* head){


    while( head != nullptr){
        std::cout << head-> data;

        //to chech if the next line is not equal to null
        if (head-> next != nullptr){
            std::cout << "->";
        }
        head = head ->next;
    }


    std::cout << std::endl;
}



template <typename T>
void sllInsertHead(T newData, singlelist<T>** currentHead){


    //allocate the memory for the new node
    singlelist<T>* newNode = new singlelist<T>;
    // put new data in the new node
    newNode ->data = newData;
    //set next of the new node to point to the previous head
    newNode -> next = *currentHead;
    //reset head tp point to the new node
    *currentHead = newNode;
}
template <typename T>
void sllGeneralInsertion(T newData, singlelist<T> * prevNode){

```

```
49
50      //check if it is the head node (previous node is null)
51      if(prevNode==nullptr){
52          std::cout <<"Previous node cannot be null." << std::endl;
53          return;
54      }
55
56
57      //allocate data
58      singlelist<T>* newNode = new singlelist<T>;
59      //store data in new mode
60      newNode -> data = newData;
61      //point nre node to to the previous node was pointing to
62      newNode -> next = prevNode -> next;
63      //point previous node to the new
64      prevNode -> next = newNode;
65  }
66
67
68  template <typename T>
69  void sllInsertEnd(T newData, singlelist<T>** head){
70
71
72  //dereferemce to the head node
73      singlelist<T>* currentNode = *head;
74
75
76  //allocate new node
77      singlelist<T>* newNode = new singlelist<T>;
78
79
80  //store data in new node
81      newNode ->data = newData;
82
83
84  //point next of new node to null
85      newNode-> next = nullptr;
86
87
88  //check if the list is emptu
89
90
```

```
91  if(currentNode == nullptr){
92      currentNode == newNode;
93      return;
94  }
95
96  //traverse the list until nxt of the node is null
97  while(currentNode-> next != nullptr){
98      //itteration foing to one node to another
99      currentNode = currentNode -> next;
100 }
101 currentNode -> next = newNode;
102 }
103
104
105 template <typename T>
106 void sllDelete(T findData, singlelist<T>** head){
107
108 //chech if there is a list
109     if(*head == nullptr) return;
110
111
112     singlelist<T>* currNode = *head;
113     singlelist<T>* prevNode = nullptr;
114
115
116     //serarch for the node
117     while(currNode != nullptr && currNode->data != findData){
118         prevNode = currNode;
119         currNode = currNode->next;
120
121     }
122
123
124     //if data is not found
125     if(currNode == nullptr)return;
126
127
128     //delete the head node (the headnode is the find dara)
129     if (prevNode == nullptr){
130         *head = currNode ->next;
131
132     }else{
133
134
```

```
135             prevNode -> next = currNode -> next;
136         }
137
138         delete currNode;
139     }
140
141
142     #endif
```

```
Linked List Traversal: 1->2->3
Newly Inserted Head: 0->1->2->3
Newly inserted after a node of linked list:
0->1->3->2->3
0->1->3->4->2->3
0->1->3->4->2->3->10
```

6.

Main cpp

```
1   #include <iostream>
2   #include "singly_ll.h"
3
4   int main() {
5       // this line creates a empty kinked list
6       singlelist<int> *head = new singlelist<int>;
7       singlelist<int> *second = new singlelist<int>;
8       singlelist<int> *third = new singlelist<int>;
9
10      head -> data = 1;
11      head -> next = second;
12
13      head -> data = 2;
14      head -> next = third;
15
16      head -> data = 3;
17      head -> next = nullptr;
18
19      std::cout <<"Linked List Traversal: ";
20      listTraversal(head);
21
22      std::cout<< "Newly Inserted Head: ";
23      sllInsertHead(0,&head);
24      listTraversal(head);
25
26      std::cout<< "Newly inserted after a node of linked list: \n";
27      sllGeneralInsertion(3,head->next);
28      listTraversal(head);
29
30      sllGeneralInsertion(4,head->next->next);
31      listTraversal(head);
32
33      sllInsertEnd(10,&head);
34      listTraversal(head);
35
36      std::cout <<"deleting a node: \n";
37      sllDelete(10,&head);
38      listTraversal(head);
39
40      std::cout <<"Deleting the list \n";
41      listTraversal(head);
42      sllDeleteList(&head);
43      listTraversal(head);
44
45      return 0;
46  }
```

```cpp
#ifndef SINGLY_LL_H // header file name
#define SINGLY_LL_H
#include<iostream>
template <typename T> // enables the class accept any data type


class singlelist{
    public:
    T data; //data of the node
    singlelist<T> *next = nullptr; //stores address of next node
};
template <typename T>
void listTraversal(singlelist<T>* head){


    while( head != nullptr){
        std::cout << head-> data;
        //to check if the next line is not equal to null
        if (head-> next != nullptr){
            std::cout << "->";
        }
        head = head ->next;
    }
    std::cout << std::endl;
}

template <typename T>
void sllInsertHead(T newData, singlelist<T>** currentHead){


    //allocate the memory for the new node
    singlelist<T>* newNode = new singlelist<T>;
    // put new data in the new node
    newNode ->data = newData;
    //set next of the new node to point to the previous head
    newNode -> next = *currentHead;
    //reset head tp point to the new node
    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsertion(T newData, singlelist<T> * prevNode){


    //check if it is the head node (previous node is null)
    if(prevNode==nullptr){
        std::cout <<"Previous node cannot be null." << std::endl;
        return;
    }
```

```
50  //allocate data
51          singlelist<T>* newNode = new singlelist<T>;
52          //store data in new mode
53          newNode -> data = newData;
54          //point nre node to to the previous node was pointing to
55          newNode -> next = prevNode -> next;
56          //point previous node to the new
57          prevNode -> next = newNode;
58  }
59
60      template <typename T>
61      void sllinsertEnd(T newData, singlelist<T>** head){
62
63          //dereferemce to the head node
64          singlelist<T>* currentNode = *head;
65          //allocate new node
66          singlelist<T>* newNode = new singlelist<T>;
67          //store data in new node
68          newNode ->data = newData;
69          //point next of new node to null
70          newNode-> next = nullptr;
71          //check if the list is emptu
72          if(currentNode == nullptr){
73              currentNode == newNode;
74              return;
75          }
76          //traverse the list until nxt of the node is null
77          while(currentNode-> next != nullptr){
78              //itteration foing to one node to another
79              currentNode = currentNode -> next;
80          }
81          currentNode -> next = newNode;
82      }
83
84      template <typename T>
85      void sllDelete(T findData, singlelist<T>** head){
86
87          //check if there is a list
88          if(*head == nullptr) return;
89
90          singlelist<T>* currNode = *head;
91          singlelist<T>* prevNode = nullptr;
92          //serarch for the node
93          while(currNode != nullptr && currNode->data != findData){
```

```
94                  prevNode = currNode;
95                  currNode = currNode->next;
96
97              }
98              //if data is not found
99              if(currNode == nullptr)return;
100             //delete the head node (the headnode is the find dara)
101             if (prevNode == nullptr){
102                 *head = currNode ->next;
103             }else{
104                 prevNode -> next = currNode -> next;
105             }
106             delete currNode;
107         }
108
109         template <typename T>
110         void sllDeleteList(singlelist<T> ** head){
111             singlelist<T>* currentNode = *head;
112             while (currentNode != nullptr){
113                 singlelist<T>* temp = currentNode;
114                 currentNode = currentNode->next;
115                 delete temp;
116             }
117             *head = nullptr;
118         }
119     #endif
```

```
Linked List Traversal: 3
Newly Inserted Head: 0->3
Newly inserted after a node of linked list:
0->3->3
0->3->3->4
0->3->3->4->10
deleting a node:
0->3->3->4
Deleting the list
0->3->3->4
```

7.

```cpp
#include <iostream>
#include "double_LL.h"

int main(){

    //'C' 'P' 'E'
    //CREATION OF HEAD NODE:
    doubleList<char>* head = new_Node('C');
    //CREATION OF OTHER NODES:
    doubleList<char>* second = new_Node('P');
    doubleList<char>* third = new_Node('E');

    //LINK THE NODES
    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    std::cout<<"Initial Traversal"<<std::endl;
    dllTraversal(head);

    std::cout<<"Inserting at the head"<<std::endl;
    dllInsertHead('G',&head);
    dllTraversal(head);

    std::cout<<"\nNewly inserted after a node of Linked list: \n";
    dllGeneralInsert('N',head->next);
    dllTraversal(head);

    std::cout<<"\ndeleting a node: \n";
    dllDelete('N',&head);
    dllTraversal(head);

    std::cout<<"\ndeleting the list: \n";
    dllTraversal(head);
    dllDeleteList(&head);

    std::cout << " " <<std::endl;
    dllTraversal(head);

}
```

```cpp
#ifndef DOUBLE_LL_H

template <typename T>
class doubleList{
    public:
        T data;
        doubleList<T>* next = nullptr;
        doubleList<T>* prev = nullptr;
};




template <typename T>
doubleList<T>* new_Node(T newData){
    //dynamically allocate memory for a new node
    doubleList<T>* newNode = new doubleList<T>;

    //Store data in the node:
    newNode->data = newData;

    //since its a new node, it will point to null
    newNode ->next = nullptr;
    newNode->prev = nullptr;

    return newNode;

}
//Traversal
template <typename T>
void dllTraversal(doubleList<T>* currentNode){
    doubleList<T>* tail;

    //traversal to the end
    std::cout<< "Forward traversal\n";
    while(currentNode!= nullptr){
        //prints the data
        std::cout<< currentNode->data<<" ";
        //save the current node as the last visited node
        tail = currentNode;
        //move to the next node
        currentNode = currentNode->next;
    }

```

```
45      std::cout<<std::endl;
46      //reverse traversal
47      std::cout << "Reverse travesal \n";
48      while(tail != nullptr){
49          std::cout<< tail->data <<" ";
50          tail = tail->prev;
51      }
52  }
53
54
55  //Insertion
56  template <typename T>
57  void dllInsertHead(T newData, doubleList<T>** currentHead){
58
59      //create a new node
60      doubleList<T>* newNode = new_Node(newData);
61
62      //SET THE connect the new node to point to the current head
63      newNode->next = *currentHead;
64
65      //current head will be pointed to the new node
66      (*currentHead)->prev = newNode;
67
68      //update the head pointer
69      *currentHead = newNode;
70
71  }
72
73  //insetion at the head
74
75  template <typename T>
76  void dllInsertEnd(T newData, doubleList<T>* currentHead){
77
```

```
78      //create a new node
79      doubleList<T>* newNode = new_node(newData);
80
81      //traverse until the last node
82      while(currentHead->next != nullptr){
83          currentHead = currentHead ->next;
84      }
85
86      //connect the last node to the new node
87      currentHead ->next = newNode;
88
89
90      //connect the prev node to the new node
91      newNode->prev = currentHead;
92
93  }
94
95  template <typename T>
96  void dllGeneralInsert(T newData, doubleList<T>* prevNode)
97  {
98      if (prevNode == nullptr) {
99          std::cout << "Previous node cannot be null.\n";
100         return;
101     }
102
103     doubleList<T>* newNode = new_Node(newData);
104
105     // Link the new node
106     newNode->next = prevNode->next;
107     newNode->prev = prevNode;
108
109     // If there is a node after prevNode, fix its prev pointer
110     if (prevNode->next != nullptr)
111         prevNode->next->prev = newNode;
112
113     // Finish the insertion
114     prevNode->next = newNode;
115 }
116
117 template<typename T>
118 void dllDelete(T findData, doubleList<T>** head){
119     if (*head == nullptr)
120         return;
121
```

```
122     doubleList<T>* currNode = *head;
123 
124     while (currNode != nullptr && currNode->data != findData)
125         currNode = currNode->next;
126 
127     if (currNode == nullptr)
128         return;
129 
130     if (currNode->prev != nullptr)
131         currNode->prev->next = currNode->next;
132 
133     else
134         *head = currNode->next;
135 
136     if (currNode->next != nullptr)
137         currNode->next->prev = currNode->prev;
138 
139     delete currNode;
140 }
141 
142 template <typename T>
143 void dllDeleteList(doubleList<T>**head){
144     doubleList<T>* currNode = *head;
145 
146     while(currNode != nullptr){
147         doubleList<T>* temp = currNode;
148         currNode = currNode-> next;
149         delete temp;
150     }
151     *head = nullptr;
152 }
153 
154 #endif
```

```
Forward traversal
G C P E
Reverse travesal
E P C G
deleting the list:
Forward traversal
G C P E
Reverse travesal
E P C G
Forward traversal

Reverse travesal
```

Lesson learned

While working on the code, I learned more about how programming works. One thing I noticed is that programming is case-sensitive. This means that uppercase and lowercase letters must be used correctly. A small mistake, like using a capital letter instead of a lowercase one, can cause the program to stop working. I also learned that some functions can change the original data by using references. This helped me understand how different parts of a program can share and update the same data. Another thing I learned is the importance of managing memory. If I create something new in the program, I need to make sure I remove it when it is no longer needed. Doing this helps avoid memory leaks and keeps the program from using more memory than necessary. I also learned that checking if data is valid before using it is very important. These safety checks can prevent errors and stop the program from crashing.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

main.cpp

```cpp
#include <iostream>
#include <string>
#include "double_LL.h"

int main() {

    // CREATION OF HEAD NODE:
    doubleList<std::string>* head = new_node(std::string("Song_A"));

    // CREATION OF OTHER NODES:
    doubleList<std::string>* second = new_node(std::string("Song_B"));
    doubleList<std::string>* third = new_node(std::string("Song_C"));

    // LINK THE NODES
    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    std::cout << "Initial Traversal" << std::endl;
    dllTraverse(head);

    std::cout << "Inserting at the head" << std::endl;
    dllInsertHead(std::string("Song_Intro"), &head);
    dllTraverse(head);

    std::cout << "General Insertion (Inserting 'Song_X' after 'Song_B'):" << std::endl;
    dllInsertAfter(second, std::string("Song_X"));
    dllTraverse(head);

    std::cout << "Inserting at the end ('Song_Z'):" << std::endl;
    dllInsertEnd(std::string("Song_Z"), head);
    dllTraverse(head);

    std::cout << "General Deletion (Deleting 'Song_X'):" << std::endl;
    dllDeleteNode(&head, second->next);
    dllTraverse(head);

    std::cout << "Deleting from the end:" << std::endl;
    dllDeleteEnd(&head);
    dllTraverse(head);

    return 0;
}
```

double_ll.h

```cpp
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H

#include <iostream>
#include <string>

template <typename T>
class doubleList {
public:
    T data;
    doubleList<T>* next = nullptr;
    doubleList<T>* prev = nullptr;
};

// ====================
// creating a node
// ====================
template <typename T>
doubleList<T>* new_node(T newData) {
    doubleList<T>* newNode = new doubleList<T>();
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

// ====================
// traversal
// ====================
template <typename T>
void dllTraverse(doubleList<T>* currentNode) {
    doubleList<T>* tail = nullptr;
    while (currentNode != nullptr) {
        // prints the data
        std::cout << currentNode->data << " ";
        // save the current node as the last visited node
        tail = currentNode;
        // move to the next node
        currentNode = currentNode->next;
    }

```

```cpp
    std::cout << std::endl;

    // reverse traversal
    std::cout << "Reverse Traversal: ";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl;
}

// ===================
// insertion at the head
// ===================
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {
    // create a new node
    doubleList<T>* newNode = new_node(newData);

    // SET the new node to point to the current head
    newNode->next = *currentHead;

    // current head will be pointed to the new node
    if (*currentHead != nullptr) {
        (*currentHead)->prev = newNode;
    }

    // update the head pointer
    *currentHead = newNode;
}

// ===================
// general insertion
// ===================
template <typename T>
void dllInsertAfter(doubleList<T>* prevNode, T newData) {
    if (prevNode == nullptr) {
        std::cout << "The given previous node cannot be NULL" << std::endl;
        return;
    }

```

```cpp
    // create a new node
    doubleList<T>* newNode = new_node(newData);

    // set new node's next to prevNode's next
    newNode->next = prevNode->next;

    // connect prevNode's next to newNode
    prevNode->next = newNode;

    // connect newNode's prev to prevNode
    newNode->prev = prevNode;

    // connect next node's prev to newNode if it exists
    if (newNode->next != nullptr) {
        newNode->next->prev = newNode;
    }
}

// ====================
// insertion at the end
// ====================
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead) {
    // create a new node
    doubleList<T>* newNode = new_node(newData);

    if (currentHead == nullptr) {
        return;
    }

    // traverse until the last node
    doubleList<T>* currentNode = currentHead;
    while (currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }

    // connect the last node to the new node
    currentNode->next = newNode;

    // connect the prev node to the new node
    newNode->prev = currentNode;
}
```

```
126
127 // ====================
128 // general deletion (deletes a given node)
129 // ====================
130 template <typename T>
131 void dllDeleteNode(doubleList<T>** currentHead, doubleList<T>* delNode) {
132     if (*currentHead == nullptr || delNode == nullptr) return;
133
134     // if node to be deleted is head node
135     if (*currentHead == delNode) {
136         *currentHead = delNode->next;
137     }
138
139     // change next only if node to be deleted is NOT the last node
140     if (delNode->next != nullptr) {
141         delNode->next->prev = delNode->prev;
142     }
143
144     // change prev only if node to be deleted is NOT the first node
145     if (delNode->prev != nullptr) {
146         delNode->prev->next = delNode->next;
147     }
148
149     delete delNode;
150 }
151
152 // ====================
153 // delete from end
154 // ====================
155 template <typename T>
156 void dllDeleteEnd(doubleList<T>** currentHead) {
157     if (*currentHead == nullptr) return;
158
159     doubleList<T>* temp = *currentHead;
160
161     if (temp->next == nullptr) {
162         *currentHead = nullptr;
163         delete temp;
164         return;
165     }
166
167     while (temp->next != nullptr) {
168         temp = temp->next;
169     }
170
```

```
171         temp->prev->next = nullptr;
172         delete temp;
173     }
174
175     #endif
```

Summary of Lessons Learned

In this HOA, I learned how to use doubly linked lists with C++ templates. I learned that each node stores data and has two pointers, **next** and **prev**, to connect it to the next and previous nodes. I also learned how to use **new** to create nodes and **delete** to remove them and free memory. I also understood how to move through the list in both forward and backward directions using these pointers.

Analysis of the Supplementary Activity

The supplementary activity helped me understand how doubly linked lists can be used in a real program by creating a song playlist. Instead of storing simple values, each node stored a song title using **std::string**. I practiced adding songs at the beginning, middle, and end of the playlist, removing songs, and moving through the playlist using the next and previous pointers. This made it easier to see how doubly linked lists work in everyday applications.

Concluding Statement / Feedback

I think I did a good job completing this activity because I understood how nodes are added, connected, and removed from the list. I still need more practice with pointer management, especially when updating pointers after adding or deleting nodes. I also want to improve my understanding of edge cases, such as when the list is empty or has only one node.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I used chatgpt as idea generation for the code that i want to plug in, to modify it and grammarly is to correct my grammar.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.